Welcome to JMB's documentation

For full documentation visit mkdocs.org. By Jean-Michel Bruel.

Warning

Work in progress. This document is not complete.

Commands

- `mkdocs new [dir-name]` - Create a new project.
- `mkdocs serve` - Start the live-reloading docs server.
- `mkdocs build` - Build the documentation site.
- `mkdocs -h` - Print help message and exit.

Project layout

```
mkdocs.yml    # The configuration file.
docs/
    index.md  # The documentation homepage.
    ...       # Other markdown pages, images and other files.
```

BASAALT: Behaviour Analysis & Simulation All Along systems Life Time

This projects aims at developing the BASAALT methods

Ongoing efforts

- This repo for the future online documentation of the language
- A repo for all the current material: https://github.com/CoCoVaD/basaalt, including:
- The PDF documentation
- The Xtext ongoing development of an eclipse editor
- the plantUML diagrams of the concepts
- The Master project on a vscode editor that is about to start :blink:

Introduction

Context & Objective of the Document

This documentation presents the principles of **BASAALT** (Behaviour Analysis & Simulation All Along systems Life Time), a model-based systems engineering (MBSE) approach, and the notions, the syntax and examples of the FOrmal Requirements Modelling Language (FORM-L) that supports the formal and semi-formal aspects of BASAALT. Both are illustrated with extensive examples.

Rationale

At the beginning of the MODRIO project, the plan was first to list the desirable features for a rigorous language suitable for the modelling, simulation and formal analysis of large and complex socio-technical systems (i.e., systems integrating human and technological aspects), cyber-physical systems (i.e., systems integrating physics, computing and networking) and large, distributed systems of systems (i.e., dynamic sets of multiple, more or less independent but interacting systems) all along their lifecycles, then to perform a survey of the existing languages, and finally to select the most appropriate one. Unfortunately, although the survey identified several languages suitable for cyber systems (i.e., systems based exclusively on digital electronics, computing and networking), none was found that also covered the human and physical aspects (see DuriaudY2007) and AzzouziE2019). Thus, the list of desirable features gradually morphed into the BASAALT approach and the FORM-L language. As the engineering of large and complex systems covers a very wide spectrum that would be difficult and inconvenient to address with a single method and a single language, BASAALT and FORM-L are centered on and deal with dynamic phenomena, even though they can address, with limited capability and ease, other aspects such as geometry and topology. They have four main objectives:

- To support the rigorous and unambiguous modelling of constraints on dynamic phenomena, with variables (representing continuous or discrete states), events (representing facts occurring in time but the duration of which is neglected), properties (such as assumptions, objectives and requirements) and objects (representing real world entities such as systems, subsystems, human agents or the physical environment).
- To support the engineering of systems all along their lifecycle with successive refinements to keep track of a system and its constituents as their engineering progresses.
- To support the coordination necessary to the engineering of large systems, with contracts (formal interfaces between two or more systems or subsystems), bindings (formal

FORM-L Notations

Warning

Work in progress. This document is not complete.

Notation for Examples

For easy identification, illustrative examples are given within boxes with a light grey background (see voltage-property).

FORM-L models are written using this `font`. Comments are written with a normal font.

Names follow the *CamelCase* convention, where names based on multiple words are written without separators, the beginning of each word (except the first one) being marked by a single capital letter. Model and template names begin with a capital letter, whereas instance names begin with a lowercase letter. For example:

FORM-L	Explanation
Voltage **mpsVoltage**	where `Voltage` is a type (i.e., a template), and `mpsVoltage` a variable (i.e., an instance)
BpsIntroduction	the name of a model

By convention, event names begin with an `e`. Also, the use of a predefined FORM-L standard library defining physical types and constants of various types is assumed.

For example:

FORM-L	Explanation
1*s	1 second
10*V	10 volts (constants representing standard units begin sometimes with a capital letter to conform to the international standard notation)
100*ms	100 milliseconds
1000*kW	1000 kilowatts

To facilitate legibility and understanding, the examples are shown with syntactic highlighting as can be provided by syntactic editors such as jEdit or Notepad++:

- General purpose keywords are in *deep blue italics begin end*
- The nature of a FORM-L item and the types defined by the standard library are in *deep italics: Property, Class, Boolean, Mass*

References

[Bettini16] Bettini L., “Implementing Domain-Specific Languages with Xtext and Xtend”, PACKT Publishing, 2016.

[DuriaudY2007] Y., “EuroSysLib – Deliverable 7.1b – Etude des Langages de Spécifications de Propriétés”, EDF technical report H-P1A-2007-01326-FR, 2007.

[AzzouziE2019] E., Jardin A., Mhenni F., “A Survey on Systems Engineering Methodologies for Large Multi-Energy Cyber-Physical Systems” 13th Annu. Int. Syst. Conf. SysCon 2019 - Proc., April 2019.